

Abstract Data Structure

Unit-06

An abstract data structure (ADS) is a theoretical concept within computer science that specifies how data is organized, stored, and manipulated in a computer system, without delving into the specifics of its implementation. It provides the necessary operations to access and manipulate the data it contains, focusing on what operations are performed rather than how these operations are implemented. Abstract data structures form the foundation for designing and analyzing algorithms and are crucial for efficient data processing and storage.

Arrays: A collection of elements identified by index or key, storing elements of the same type. Arrays allow fast access to elements but can be inefficient for insertion and deletion operations.

Linked Lists: Consist of nodes where each node contains data and a reference (or link) to the next node in the sequence. This structure allows for efficient insertion and deletion of elements.

Stacks: A collection that follows the Last In, First Out (LIFO) principle, where the most recently added element is the first to be removed. Stacks are useful for tasks such as reversing data or parsing expressions.

Queues: A collection that follows the First In, First Out (FIFO) principle, where the first element added is the first to be removed. Queues are used in scenarios like task scheduling and buffering.

Trees: A hierarchical data structure with a root value and subtrees of children with a parent node, represented as a set of linked nodes. Trees are used in databases, file systems, and for efficient searching and sorting.

Graphs: A collection of nodes (or vertices) connected by edges. Graphs can represent networks, such as social networks or transportation grids, and are used in various algorithms, including searching and pathfinding.

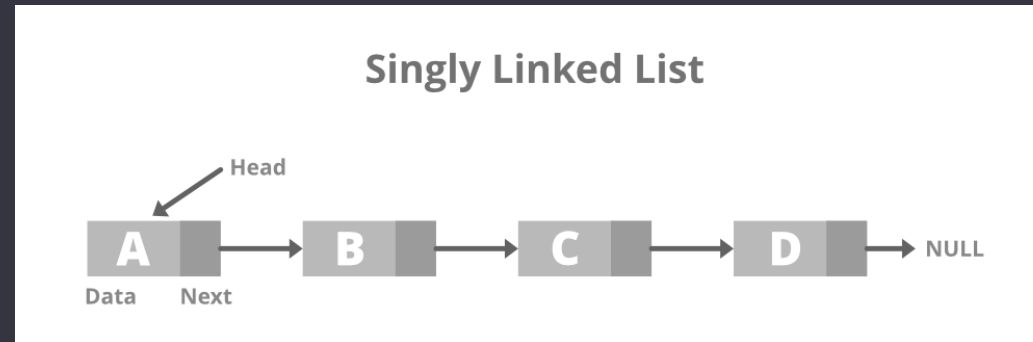
Hash Tables: Implements an associative array, a structure that can map keys to values. Hash tables are highly efficient for lookup and insertion operations and are widely used in database indexing.

Linked Lists

Linked lists are fundamental data structures that consist of a series of nodes, where each node contains data and a reference (or link) to the next node in the list. The structure allows for efficient insertion and deletion of elements because these operations do not require the elements to be contiguous in memory. Linked lists come in several varieties, including singly linked lists, doubly linked lists, and circular linked lists, each offering different advantages for specific situations.

Singly Linked Lists

In a singly linked list, each node points to the next node in the sequence, with the last node pointing to null, indicating the end of the list. This structure is simple and uses less memory per node (since there's only one pointer), but it can only be traversed in one direction.



Use Cases for Singly Linked Lists:

Implementation of stacks and queues: Singly linked lists are well-suited for these data structures since elements are added and removed from the ends.

Lightweight and simple storage needs: When the data structure's complexity and memory usage are critical factors, singly linked lists offer a straightforward and efficient solution.

Singly Linked Lists: Implementation

Can you implement **insert** and **delete**?

```
struct Node {  
    int data;  
    Node* next;  
  
    Node(int data) {  
        this->data = data;  
        this->next = nullptr;  
    }  
};
```

insert:

- If the current node is last node, assign new node to
 - this->next
- If the current node is some intermediate nodes, traverse to until you find the last node, and now assign the new node

delete:

- Check if the value matches
- If matched: delete the node and join two neighboring nodes if the node is intermediate
- If matched: delete the node and assign this->next a nullptr to the previous node if the node to be deleted is the last node

Singly Linked Lists: Implementation

```
struct SinglyLinkedList {
    Node* head;
    SinglyLinkedList(){
        head = nullptr;
    }
    void insert(int);
    void deleteNode(int);
};
```

// Function to insert a new node at the end

```
void insert(int newData) {
    Node* newNode = new Node(newData);
    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

```
bool deleteNode(int valueToDelete) {
    Node* temp = head;
    Node* prev = nullptr;

    // If head needs to be removed
    if (temp != nullptr && temp->data == valueToDelete) {
        head = temp->next;
        delete temp;
        return true;
    }
```

```
    // Search for the node to be deleted
    while (temp != nullptr && temp->data != valueToDelete) {
        prev = temp;
        temp = temp->next;
    }
```

```
    // If node was not found
    if (temp == nullptr) return false;
```

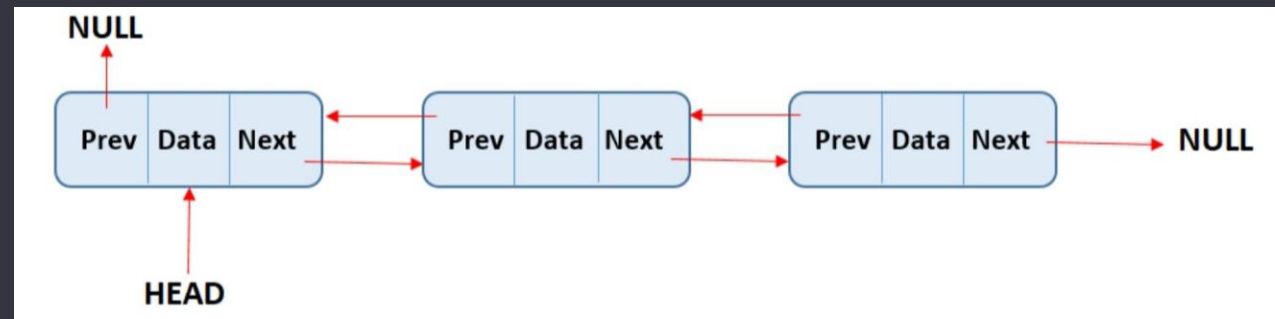
```
    // Unlink the node from the list and delete it
    prev->next = temp->next;
    delete temp;
    return true;
```

```
}
```



Doubly Linked Lists

Doubly linked lists allow each node to have two links: one pointing to the next node and one to the previous node. This bidirectional traversal capability makes some operations more efficient, such as deletion of a node when only the node to be deleted is given.



Use Cases

Navigational systems: The ability to move forward and backward through the list is useful in applications where users need to navigate through a set of items, such as a web browser's history.

Implementing data structures that require both ends access: Doubly linked lists are ideal for implementing data structures like deques (double-ended queues) that require efficient operations on both ends of the sequence.

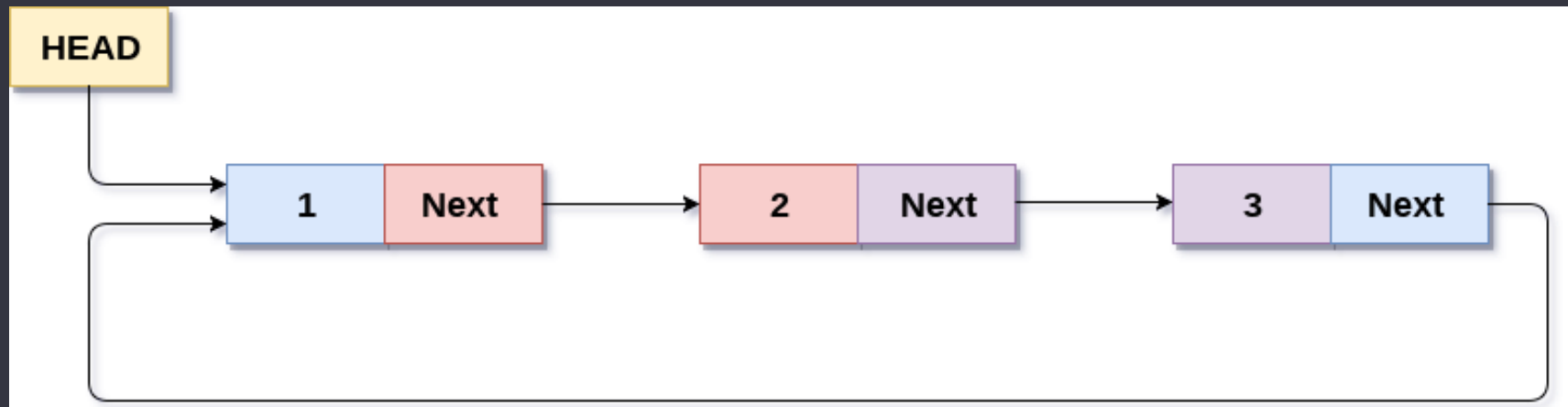
Double Linked Lists: Implementation

```
struct Node {  
    int data;  
    Node* next;  
    Node* prev;  
  
    Node(int data) : data(data), next(nullptr), prev(nullptr) {}  
};
```

How do we implement insert, delete?

Circular Linked Lists

Circular linked lists are a variation of linked lists where the last node points back to the first node, forming a circle. This can be applied to both singly and doubly linked lists. Circular linked lists are useful for cyclically accessed data, where the list needs to be looped through repeatedly without needing to find the end.

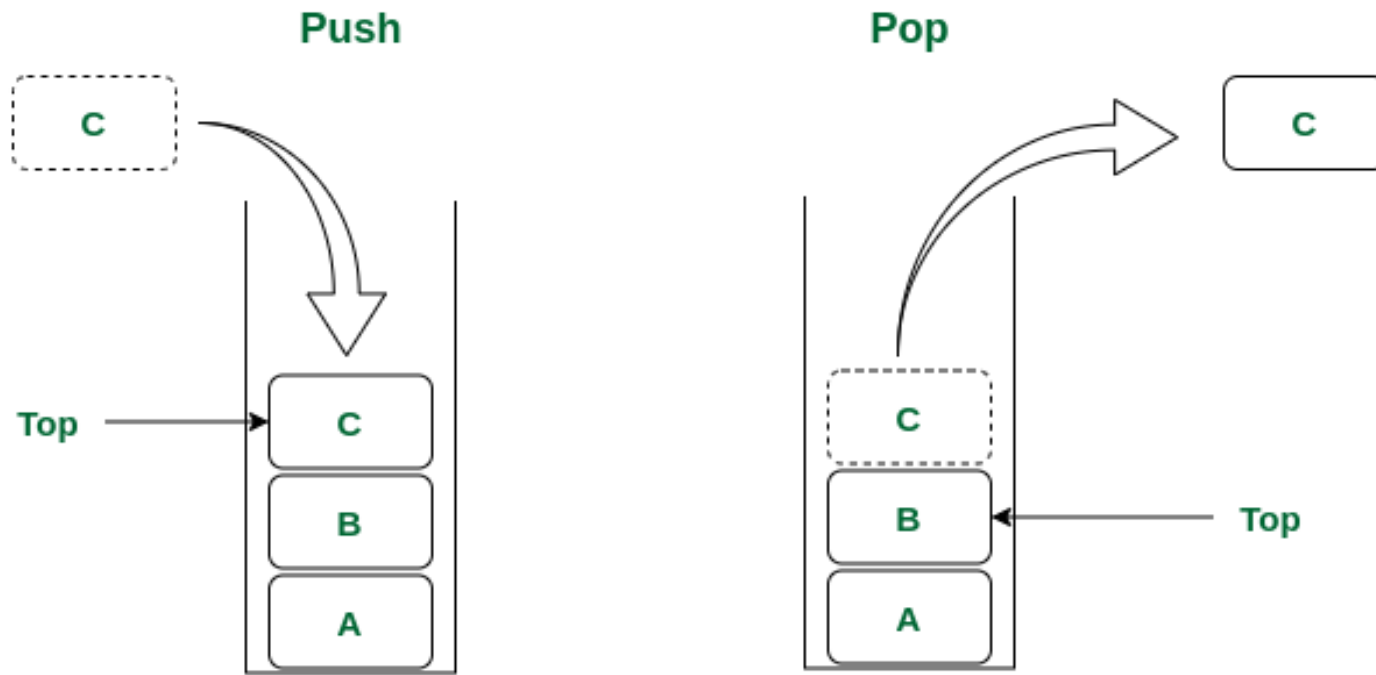


Use Cases

Implementing round-robin schedulers: In operating systems, circular linked lists can be used to manage processes in a round-robin scheduling algorithm efficiently.

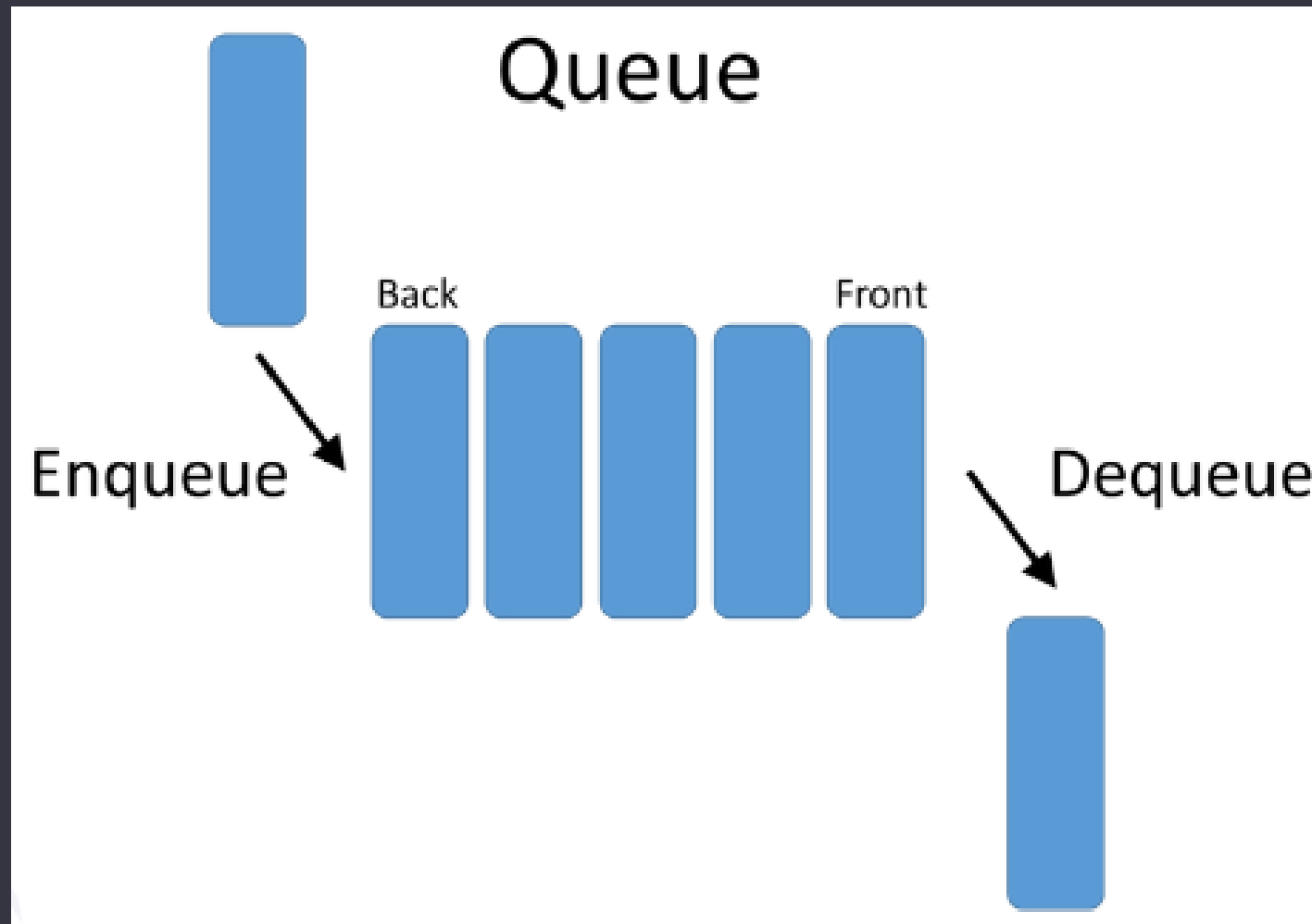
Music playlists or slideshow applications: Where the items (e.g., songs, images) need to be looped through continuously without interruption.

Can you use previous concept and implement following Stack?



Stack Data Structure

Can you use previous concept and implement following Stack?



Fun Exercise : Simulating Function Call Stack

Objective: The main goal of this exercise is to simulate the call stack mechanism that happens in the background when functions call other functions, including recursive function calls. Visualize the LIFO (Last In, First Out) nature of the call stack.

Step 1: Define the Stack Node

Each node in the stack will represent a function call, containing the function's name and any relevant parameters or local variables.

Step 2: Implement the Call Stack

Create a `CallStack` class to manage the function calls. Include methods to push a function onto the stack (simulating a function call) and pop a function from the stack (simulating returning from a function).

Step 3: Demonstrate with a Recursive Function

Use the `CallStack` to simulate a simple recursive function, such as calculating factorial, to visualize how the call stack changes with each function call.

Fun Exercise : Simulating Function Call Stack

cpp

 Copy code

```
struct StackNode {  
    string functionName;  
    map<string, string> parameters; // Use this to store parameters and their values  
    StackNode* next;  
  
    StackNode(string functionName, map<string, string> params)  
        : functionName(functionName), parameters(params), next(nullptr) {}  
};
```

Fun Exercise : Simulating Function Call Stack

cpp

 Copy code

```
class CallStack {
private:
    StackNode* top;

public:
    CallStack() : top(nullptr) {}

    void push(string functionName, map<string, string> params) {
        StackNode* newNode = new StackNode(functionName, params);
        newNode->next = top;
        top = newNode;
        cout << "Entering function: " << functionName << endl;
    }

    void pop() {
        if (top == nullptr) return;
        cout << "Exiting function: " << top->functionName << endl;
        StackNode* temp = top;
        top = top->next;
        delete temp;
    }

    ~CallStack() {
        while (top != nullptr) {
            pop();
        }
    }
};
```



Fun Exercise : Simulating Function Call Stack

cpp

 Copy code

```
int factorial(int n, CallStack& callStack) {
    callStack.push("factorial", {"n", to_string(n)});
    if (n <= 1) {
        callStack.pop();
        return 1;
    } else {
        int result = n * factorial(n - 1, callStack);
        callStack.pop();
        return result;
    }
}

int main() {
    CallStack callStack;
    int number = 5;
    int result = factorial(number, callStack);
    cout << "Factorial of " << number << " is " << result << endl;
    return 0;
}
```

Heap

The heap is a specialized tree-based data structure that satisfies the heap property. *In a heap, for any given node C , if P is a parent node of C , then the key (the value) of P is either greater than or equal to (in a max heap) or less than or equal to (in a min heap) the key of C .* The node at the "top" of the heap (the root node) contains the largest key (in a max heap) or the smallest key (in a min heap). Heaps are crucial in implementing priority queues, where the queue's element with the highest priority is always at the front.

Priority Queues For processing tasks based on priority, not order, crucial in operating systems, network routers, and simulations.

Sorting Heapsort utilizes heaps for efficient, in-place sorting, beneficial for large datasets.

Graph Algorithms Improves computational efficiency in algorithms like Dijkstra's and Prim's by selecting the next node to process.

Stream Processing Maintains top elements in data streams, useful in real-time analytics and monitoring systems.

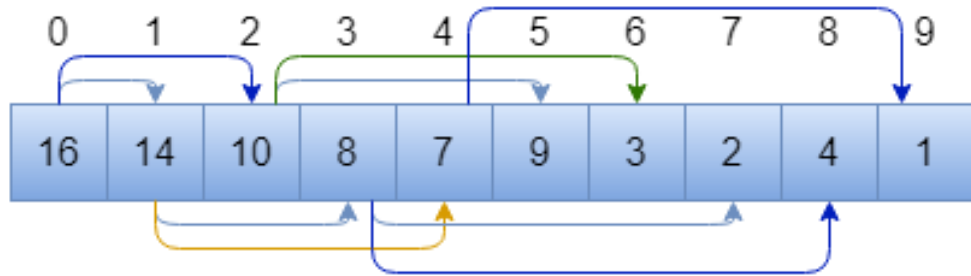
Memory Management Used in memory allocation and garbage collection in operating systems to prevent fragmentation.

Data Compression Builds Huffman trees for efficient encoding and decoding in compression algorithms.

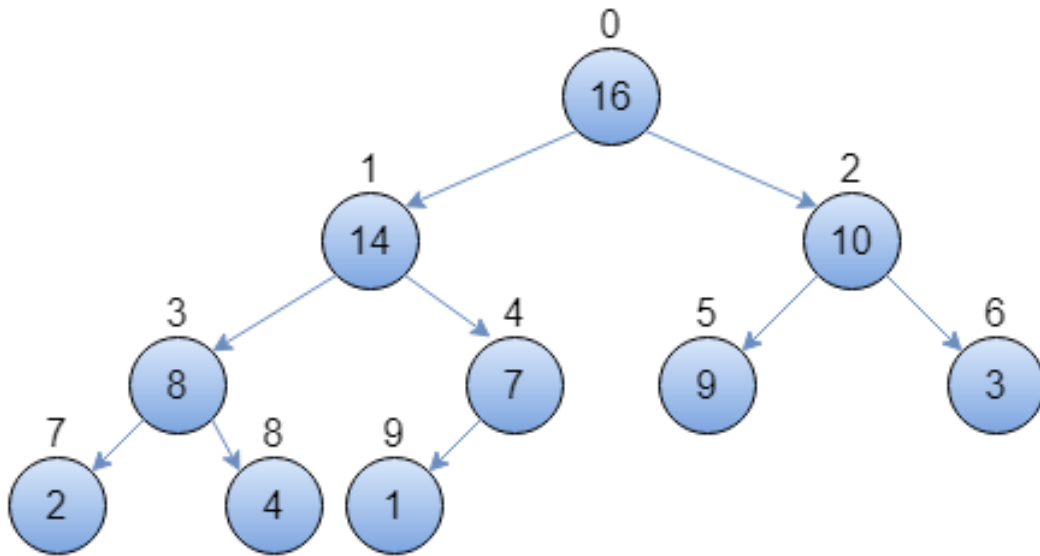
Load Balancing Distributes computing and network loads to optimize resource use and minimize response times.

Event Simulation Manages event schedules in simulations, ensuring efficient processing of upcoming events.

Heap



Array representation of a heap



Binary Tree representation of a heap

In a heap, for any given node C , if P is a parent node of C , then the key (the value) of P is either greater than or equal to (in a max heap) or less than or equal to (in a min heap) the key of C .

Max-Heapify

```
int main() {
    vector<int> arr = {12, 11, 13, 5, 6, 7};
    int n = arr.size();

    // Building max heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        MaxHeapify(arr, n, i);
    }

    cout << "Max Heap: \n";
    printArray(arr);

    return 0;
}
```

Questions...

What is the complexity of maxHeapify, delete, and insertion?

(YOU HAVE TO FIND THESE ANSWERS)

```
void maxHeapify(vector<int>& arr, int n, int i) {
    int largest = i; // Initialize largest as root
    int left = 2 * i + 1; // left child index
    int right = 2 * i + 2; // right child index

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // If largest is not root
    if (largest != i) {
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected sub-tree
        maxHeapify(arr, n, largest);
    }
}
```



The Last Stone Weight

<https://leetcode.com/problems/last-stone-weight/description/>

<https://leetcode.com/problems/last-stone-weight-ii/description/>

Preparing y'all for the job interview.....

Exercise: Meeting Room Scheduler

Background

A company has multiple meeting rooms but a very chaotic scheduling system. They've decided to implement a new system to manage meeting room bookings efficiently. Your task is to develop an algorithm that can handle the scheduling.

Problem Statement

Given an array of meeting time intervals consisting of start and end times `[[s1,e1],[s2,e2],...]` (with each meeting time `si < ei`), determine the minimum number of conference rooms required to host all the meetings.

Example

Input: `[[0, 30],[5, 10],[15, 20]]`

Output: 2

Input: `[[7,10],[2,4]]`

Output: 1

Objectives

- Implement a solution that efficiently calculates the minimum number of meeting rooms required.
- Utilize a min-heap to track the earliest ending meeting room.

Hints

- Consider sorting the intervals by start time.
- Use a min-heap to keep track of end times of meetings currently occupying rooms. Why? Because you're interested in freeing the room with the earliest end time for the next meeting.
- When iterating through the sorted meetings, if a meeting starts after the earliest ending meeting in the heap, it means we can use the same room (update the heap with the new meeting's end time). Otherwise, a new room is needed (add the meeting's end time to the heap).

Preparing y'all for the job interview.....

Exercise: Meeting Room Scheduler

Background

A company has multiple meeting rooms but a very chaotic scheduling system. They've decided to implement a new system to manage meeting room bookings efficiently. Your task is to develop an algorithm that can handle the scheduling.

Problem Statement

Given an array of meeting time intervals consisting of start and end times `[[s1,e1],[s2,e2],...]` (with each meeting time `si < ei`), determine the minimum number of conference rooms required to host all the meetings.

Example

Input: `[[0, 30],[5, 10],[15, 20]]`

Output: 2

Input: `[[7,10],[2,4]]`

Output: 1

Example Walkthrough

Given input: `[[0, 30],[5, 10],[15, 20]]`

1. **Sort by Start Time:** The meetings are already sorted.
2. **Process First Meeting** `[0, 30]`: Allocate a room. Heap = `[30]`.
3. **Process Second Meeting** `[5, 10]`: Cannot use the same room as the first meeting (it ends at 30). Allocate a new room. Heap = `[30, 10]` (min-heap ensures 10 is the root).
4. **Process Third Meeting** `[15, 20]`: The meeting starting at 5 ends at 10, so we can use this room. Update the heap with the new end time. Heap = `[20, 30]`.

Total rooms required = size of the heap = 2.

Merge K Sorted Lists

Problem Statement:

You are given an array of `k` linked-lists `**lists**`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example:

Let's consider an example where we have three sorted lists:

- List 1: `1 -> 4 -> 5`
- List 2: `1 -> 3 -> 4`
- List 3: `2 -> 6`

1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6

Find the Kth Largest Element in an Array

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Example:

Let's consider an example where we have the following array and `k`:

- **Input:** `nums = [3,2,1,5,6,4]`, `k = 2`
- **Expected Output:** `5`

Explanation:

When the array is sorted, it becomes `[1, 2, 3, 4, 5, 6]`. The 2nd largest element in this sorted array is `5`.

Additional Example:

- **Input:** `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
- **Expected Output:** `4`

Sliding Window Maximum

Problem Statement:

Given an array `nums` and an integer `k`, find the maximum element in each sliding window of size `k` as it slides from left to right across the array. You have to return an array of the maximum elements seen in each window.

Example:

Consider the array `nums = [1,3,-1,-3,5,3,6,7]` and `k = 3`.

- **Input:** `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`
- **Expected Output:** `[3,3,5,5,6,7]`

Connect n Ropes with Minimum Cost

Problem Statement:

Given `n` ropes of different lengths, you need to connect these ropes into one rope. The cost to connect two ropes is equal to the sum of their lengths. The goal is to connect all the ropes with the minimum possible cost.

Example:

Consider an array of rope lengths: `ropes = [8, 4, 6, 12]`.

- **Input:** `ropes = [8, 4, 6, 12]`
- **Expected Output:** `58`

Explanation:

1. First, connect ropes of lengths `4` and `6` (cost = 10). Now the ropes are `[8, 10, 12]`.
2. Then, connect ropes of lengths `8` and `10` (cost = 18). Now the ropes are `[18, 12]`.
3. Finally, connect the remaining two ropes (cost = 30).

The total cost is `10 + 18 + 30 = 58`. This is the minimum cost to connect all ropes.

Preparing y'all for the job interview.....

Minimum Number of Refueling Stops

Problem Statement:

A car with an initially empty fuel tank is at a starting position ``start`` and wants to reach a target position ``target``. The car can travel up to ``startFuel`` units of distance with its initial amount of fuel. Along the way, there are gas stations at positions ``stations[i][0]`` with ``stations[i][1]`` units of fuel. The car can stop at these stations to refuel. The goal is to find the minimum number of refueling stops the car must make to reach its destination. If it cannot reach the destination, return -1.

Example:

Consider a scenario where the target is 100 units away, the car starts with 10 units of fuel, and there are gas stations at positions and fuel amounts as follows:

- **Input:** ``target = 100``, ``startFuel = 10``, ``stations = [[10,60],[20,30],[30,30],[60,40]]``
- **Expected Output:** ``2``

Explanation:

1. The car starts with 10 units of fuel, enough to reach the first station at position 10.
2. At position 10, the car refuels 60 units of fuel. Now the car has 60 units of fuel.
3. The car then travels to position 30, stopping at the second and third stations but does not necessarily need to refuel at these stations to reach the fourth station. By the time it reaches position 30, it has used 20 units of fuel (from position 10 to position 30), so it has 40 units left.
4. To optimize the number of stops, the car refuels at the third station with 30 units (now has 70 units of fuel) and reaches the fourth station at position 60.
5. At position 60, the car refuels 40 units for a total of 70 units of fuel (it used 10 units to get from 30 to 60). Now, with 70 units of fuel, the car can easily reach the target at 100 without any further refueling.

The minimum number of refueling stops the car must make is ``2`` (at positions 10 and 60).

Preparing y'all for the job interview.....

.. Find the Median from a Data Stream

Problem Statement:

Design a data structure that supports the following two operations:

- ``void addNum(int num)`` - Add a number to the data structure.
- ``double findMedian()`` - Return the median of all elements so far.

Why It's Challenging:

- Requires maintaining two heaps (a max-heap for the lower half of the numbers and a min-heap for the upper half) to efficiently find the median at any time.
- Balancing the heaps after each insertion to ensure the median can be found in constant time.

Preparing y'all for the job interview.....

Find the Median from a Data Stream

Problem Statement:

Design a data structure that supports the following two operations:

- `void addNum(int num)` - Add a number to the data structure.
- `double findMedian()` - Return the median of all elements so far.

Why It's Challenging:

- Requires maintaining two heaps (a max-heap for the lower half of the numbers and a min-heap for the upper half) to efficiently find the median at any time.
- Balancing the heaps after each insertion to ensure the median can be found in constant time.

Imagine we receive a data stream of numbers: `5, 15, 10, 20, 3`.

1. Add 5

- Max-Heap: `[5]`
- Min-Heap: `[]`
- Median: `5` (Only one element, so it's the median)

2. Add 15

- Max-Heap: `[5]`
- Min-Heap: `[15]`
- Median: `(5 + 15) / 2 = 10` (Average of the two heaps' tops because we have an even number of elements)

3. Add 10

- We compare 10 with the tops. It fits between the current elements, so we adjust to maintain the heaps.
- Max-Heap: `[5, 10]` (10 moves to Max-Heap because it's part of the smaller half now)
- Min-Heap: `[15]`
- Median: `10` (Max-Heap has more elements, so its top is the median)

4. Add 20

- Max-Heap: `[5, 10]`
- Min-Heap: `[15, 20]`
- Median: `(10 + 15) / 2 = 12.5` (We have an even number of elements, so we take the average of the tops)

5. Add 3

- Max-Heap needs to adjust to include 3 as it's part of the smaller half.
- Max-Heap: `[3, 5, 10]` (After adjustment, 3 is added, and heaps are balanced if necessary)
- Min-Heap: `[15, 20]`
- Median: `10` (Max-Heap has more elements now, so its top is the median)

Preparing y'all for the job interview.....

REVIVE Twitter

Problem Statement:

Design a simplified version of Twitter. It should support the following features:

- Post a new tweet.
- Retrieve the 10 most recent tweets in the user's news feed, each time they refresh.
- Follow another user.
- Unfollow a user.

Why It's Challenging:

- Involves designing a complex system that requires efficient sorting and retrieval of tweets, often implemented using heaps to manage timelines and feeds dynamically.
- Requires comprehensive understanding of data structures to manage users, their relationships, and their tweets efficiently.

Preparing y'all for the job interview.....

Design Twitter

Problem Statement:

Design a simplified version of Twitter. It should support the following features:

- Post a new tweet.
- Retrieve the 10 most recent tweets in the user's news feed, each time they refresh.
- Follow another user.
- Unfollow a user.

Why It's Challenging:

- Involves designing a complex system that requires efficient sorting and retrieval of tweets, often implemented using heaps to manage timelines and feeds dynamically.
- Requires comprehensive understanding of data structures to manage users, their relationships, and their tweets efficiently.

Example Scenario

Let's say we have three users: Alice, Bob, and Charlie. Here's how the operations and the use of a heap might play out:

1. **Alice posts a tweet (Tweet A1).**
2. **Bob posts two tweets (Tweet B1 and B2).**
3. **Charlie follows Alice and Bob.**
4. **Charlie requests their news feed.**

For Charlie's news feed request, the system needs to gather the most recent tweets from Alice and Bob, as Charlie follows both.

- The system collects all tweets from Alice and Bob. In this scenario, let's assume it's just Tweet A1, B1, and B2.
- A min-heap is used to sort these tweets by timestamp. Since we only need the 10 most recent tweets, the heap helps efficiently sort and trim the collection to the required size.
- Assuming Tweet A1 is the oldest and B2 is the most recent, the heap allows us to efficiently retrieve B1 and B2 as the top 10 recent tweets for Charlie's feed.

Output for Charlie's News Feed Request

- **Feed:** Tweets B2, B1 (assuming the request is made after all tweets have been posted and assuming timestamps place B2 after B1).

This simplified example focuses on the core logic behind fetching a news feed and how a heap aids in managing the complexities of sorting and retrieving the most recent tweets. Real implementations would involve additional considerations, like handling large volumes of data, ensuring the system scales, and managing the dynamic nature of following/unfollowing users.